

Unit 3

Coding:-

- A coding standard gives a uniform appearance to the codes written by different engineers.
- It improves readability, and maintainability of the code and it reduces complexity also.
- It helps in code reuse and helps to detect errors easily.
- It promotes sound programming practices and increases the efficiency of the programmers.

Programming Practices:-

programming best practices are a set of informal, sometimes personal, rules that many software developers, in computer programming follow to improve software quality. Many computer programs require being robust and reliable for long periods of time, so any rules need to facilitate both initial development and subsequent maintenance of source code by people other than the original authors.

The size of a project or program has a significant effect on error rates, programmer productivity, and the amount of management needed.

Top down Approaches & Bottom up Approaches:-

Top-Down Approach is an approach to design algorithms in which a bigger problem is broken down into smaller parts. Thus, it uses the decomposition approach. This approach is generally used by structured programming languages such as C, COBOL, FORTRAN.

The drawback of using the top-down approach is that it may have redundancy since every part of the code is developed separately. Also, there is less interaction and communication between the modules in this approach.

The implementation of algorithm using top-down approach depends on the programming language and platform. Top-down approach is generally used with documentation of module and debugging code.

Bottom-Up Approach is one in which the smaller problems are solved, and then these solved problems are integrated to find the solution to a bigger problem. Therefore, it uses composition approach.

It requires a significant amount of communication among different modules. It is generally used with object oriented programming paradigm such as C++, Java, and Python. Data encapsulation and data hiding is also implemented in this approach. The bottom-up approach is generally used in testing modules.

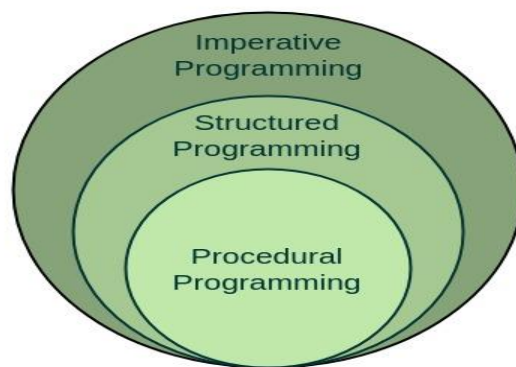
Difference between Top-Down Approach and Bottom-Up Approach:-

S.No.	Top-Down Approach	Bottom-Up Approach
1.	In this approach, the problem is broken down into smaller parts.	In this approach, the smaller problems are solved.
2.	It is generally used by structured programming languages such as C, COBOL, FORTRAN, etc.	It is generally used with object oriented programming paradigm such as C++, Java, Python, etc.
3.	It is generally used with documentation of module and debugging code.	It is generally used in testing modules.
4.	It does not require communication between modules.	It requires relatively more communication between modules.
5.	It contains redundant information.	It does not contain redundant information.
6.	Decomposition approach is used here.	Composition approach is used here.
7.	The implementation depends on the programming language and platform.	Data encapsulation and data hiding is implemented in this approach.

Structure Programming:-

a programming approach in which the program is made as a single structure. It means that the code will execute the instruction by instruction one after the other. Therefore, the instructions in this approach will be executed in a serial and structured manner. The languages that support Structured programming approach are:

- C
- C++
- Java
- C#
- ..etc



Information hiding: -

- **information hiding** is the principle of segregation of the *design decisions* in a computer program that are most likely to change, thus protecting other parts of the program from extensive modification if the design decision is changed.
- The protection involves providing a stable interface which protects the remainder of the program from the implementation (whose details are likely to change).
- Written in another way, information hiding is the ability to prevent certain aspects of a class or software component from being accessible to its

clients, using either programming language features (like private variables) or an explicit exporting policy.

Programming Style: -

- *Names: -selecting proper module and variable name.

- *Control constructs: -It is desirable that as much possible single-entry, single-exit control constructs be used.

- * Information Hiding: -If it is possible, we can use data hiding facility for hide information.

- *User defined types: -Modern languages allow user to define type like the enumerated type.

- *Nesting: -we use nesting of control structure and conditional control.

- *Module size: -A programmer should carefully examine any routine with very few statements or with too many statements

- *Module interface: -A module with a complex interface should be carefully examined

such module might not be functionality cohesive and might be implementation multiple functions.

*Program layout: -How the program is organized and proper indentation and blank space and parentheses should be used to enhance the readability of programs.

Testing: -

-The aim of program testing is to identify all defects in a program and reducing reflects in a system.

OR

-Testing perform very critical role for quality assurance and for ensuring the reliability of software during the testing the program to be tested is executed with a set of test cases and the output of program for the test cases is evaluated to determine if the program is performing as expected.

Testing Fundamental: -

Testing program involves providing the program with a set of test inputs.

-The following are some commonly used terms associated with testing.

-An error is a mistake committed by the development team during any of the development phases. The mistake might have been committed in the requirements, design, or code. An error is also sometimes referred to as a fault, a bug, or a defect.

OR

- Error refers to the difference between the actual output of software and correct output. In this interpretation, error is essentially a measure of the difference between the actual and the ideal error is also used to refer to human action those results in software containing a defect or fault.
- This definition is quite general and encompasses all the phases.

-Fault is condition that causes a system to fail in performing its required function.

A fault is the basic reason for software malfunction and is synonymous with the commonly used term bug.

-A Failure is manifestation of an error. In other words, a failure is the symptom of an error. But, mere presence of an error may not necessarily lead to a failure.

OR

-A software failure occurs if the behavior of the software is different from the specified behavior.

-Failure may be caused due to functional or performance reason.

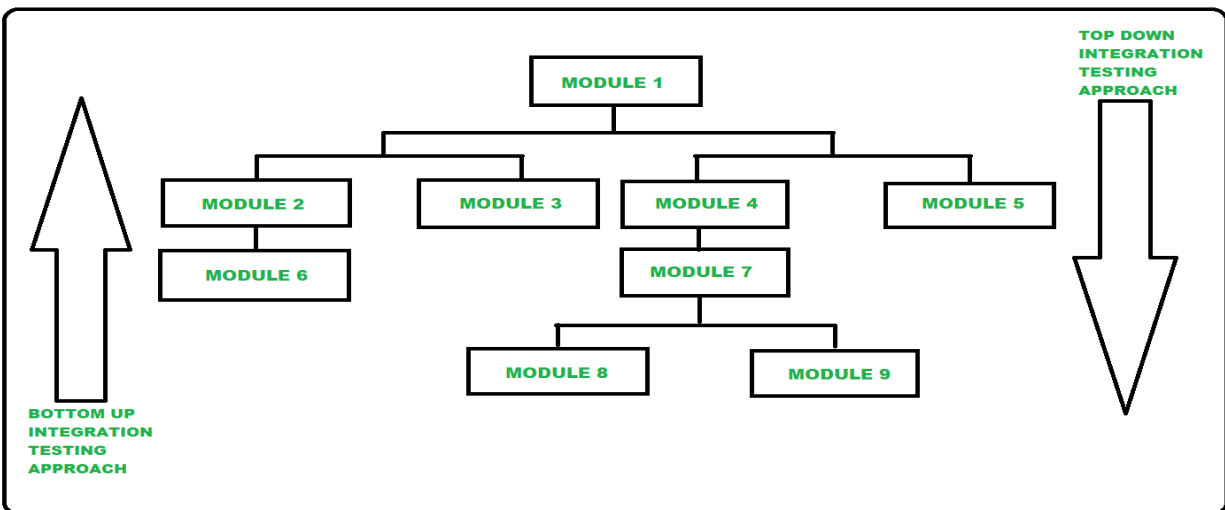
-A failure is produced only when there is a fault in the system.

-A test suite is the set of all test cases with which a given software product is tested.

Top-Down Approaches & Bottom-Up Approaches: -

Top Down Integration Testing : Top Down Integration testing which is also known as Incremental integration testing. In this Top Down approach the higher level modules are tested first after higher level modules the lower level modules are tested. Then these modules undergo for integration accordingly. Here the higher level modules refers to main module and lower level modules refers to submodules.

Bottom Up Integration Testing : Bottom Up Integration testing is another approach of Integration testing. In this Bottom Up approach the lower level modules are tested first after lower level modules the higher level modules are tested. Then these modules undergo for integration accordingly.



Difference between Top Down Integration Testing and Bottom Up Integration Testing :

S.No.	TOP DOWN INTEGRATION TESTING	BOTTOM UP INTEGRATION TESTING
01.	Top Down Integration testing is one of the approach of Integration testing in which integration testing takes place from top to bottom means system integration begins with top level modules.	Bottom Up Integration testing is one of the approach of Integration testing in which integration testing takes place from bottom to top means system integration begins with lowest level modules.
02.	In this testing the higher level modules are tested first then the lower level modules are tested and then the modules are integrated accordingly.	In this testing the lower level modules are tested first then the higher level modules are tested and then the modules are integrated accordingly.
03.	In this testing stubs are used for simulate the submodule if the invoked submodule is not developed means Stub works as a momentary replacement.	In this testing drivers are used for simulate the main module if the main module is not developed means Driver works as a momentary replacement.
04.	Top Down Integration testing approach is beneficial if the significant defect occurs toward the top of the program.	Bottom Up Integration testing approach is beneficial if the crucial flaws encounters towards the bottom of the program.
05.	In Top Down Integration testing approach the main module is designed at first then the submodules/subroutines are called from it.	In Bottom Up Integration testing approach different modules are created first then these modules are integrated with the main function.
06.	It is implemented on Structure/procedure-oriented programming languages.	It is implemented on Object-oriented programming languages.
07.	The complexity of this testing is simple.	The complexity of this testing is complex and highly data intensive.
08.	It works on big to small components.	It works on small to big components.
09.	In this approach Stub modules must be produced.	In this approach, Driver modules must be produced.

10.	In terms of cost, Top Down testing is more expensive because it requires the complete system for testing.	Bottom Up testing is less expensive as compared to Top Down because it allows early identification and resolution of the model issues.
-----	---	--

Test Cases and Test Criteria: -

A TEST CASES : is the triplet [I.S.O], where I is the data input to the system, S is the state of the system at which the data is input, and O is the expected output of the system.

1. That are good at revealing the presence of fault is central to successful testing. The reason for this is that if there is a fault in a program, the program can still provide the expected behavior for many inputs. Only for the set inputs that exercises the fault in the program will the output of the program deviate from expected behavior.
2. We would like to determine a set of test cases such that successful execution of all of them implies that there are no errors in the program.
3. While selecting test cases the primary objective is to ensure that if there is an error or fault in the program, it is exercised by one of the test cases. An ideal test case set is one that succeeds (meaning that its execution reveals no errors) only if there are no errors in the program. One possible ideal set of test cases is one that includes all the possible inputs to the program. This is often called exhaustive testing. However, it is not feasible, as even for small programs. The number of elements in the input domain can be extremely large.
4. Realistic goal for testing is to select a set of test cases that is close to ideal.
5. How should we select our test cases?

TEST CRITERIA :-

1. Test selection criteria can be used. For a given program P and its specification S. A test selection creation specifies the condition that must be satisfied by a set of test cases T. the creation become a basic for case selection.
2. Once during testing set of test cases T satisfies this creation for a program P if the execution of P with T ensures that each statement in P is executed at least once.
3. Specifying a criterion for evaluating a set of test cases and generating a set of test cases that satisfy a given criterion.
4. There are two fundamental properties for testing criterion. 1. Reliability
2. Validity

Psychology of Testing: -

Selecting testing test cases is creative activity that relies on the tester. Due to this reason psychology of the person performing the testing becomes becomes important. The aim of testing is often to demonstrate that a program works by showing that it has no errors.

The basic purpose of testing is to detect errors that may be present in the program.

Regressing Testing: -

This type of testing is require when the system being tested is an up gradation of an already existing system to fix some bugs or enhance functionality, performance,etc.

Regression testing is the practice of running an old test suite after each change to the system or after each bug fix to ensure that no new bug has been introduced due to the change or the bug fix.

If only a few statements are changed, then the entire test suite need not be run- only those test cases that test the functions that are likely to be affected by the change need to be run.

A software is changed to fix a detected bug, regression testing is carried out.

Functional Testing: -

FUNCTIONAL TESTING (BLACK BOX TESTING) :

In functional testing the structure of the program is not considered. Test cases are decided solely on the basis of the requirements or specification of the program or module. And the internals of the module or the program are not considered for selection of test cases.

Function is also called "Black Box Testing"

Software testing techniques where by the tester does not know the internal working of the item being tested. For example, in a black box test on software design the tester only knows the inputs and what the expected outcomes should be and not how the program arrives at those outputs.

The tester does not ever examine the programming code and does not have any further knowledge of the program other than its specification.

The Advantage of this type of testing includes :

- The test is unbiased because the designer and the tester are independent of each other.
- The tester does not need knowledge of any specific programming languages.
- The test is done from the point of view of the user, not the designer.

- Test cases can be designed as soon as the specification are complete.

The disadvantage of this type of testing include :

- The test can be redundant if the software designer has already run a test case.
- The test cases are difficult to design.
- Testing every possible input stream is unreliable because it would take a inordinate amount of time therefore many program paths will go untested.

In black box testing, test cases are designed on the basis of input/output and values and no knowledge of design or code is required.

There are two main approaches to design black-box test cases are given below.

1. Equivalence class partitioning
2. Boundary value analysis
3. Cause effect graphics

1.Equivalence Class Partitioning

Because we cannot do exhaustive testing. The next natural approach is to divide the domain of all the inputs into a set of equivalence classes, so that if any test in an equivalence class succeeds, then every test in that class will succeed.

In this approach, the domain of input values to a program partitioned into asset of equivalence classes. The partitioning is done such that the behavior of the program is similar to every input data belonging to the same equivalence class.

The main idea behind defining the equivalence classes is that testing the code with one value belonging to an equivalence class is as good as testing the software with any other values belonging to that equivalence class. Equivalence classes for a software can be designed by examining both the input and output data. The following are some guidelines for designing the equivalence classes :

If the input data values to a system can be specified by a range of values, then one valid and two invalid equivalence classes need to be defined.

- some domain the one Equivalence Class for the valid input value and another Equivalence Class for the invalid input value should be define for example,if the valid Equivalence Class are {A,B,C} then the invalid Equivalence Class is $U - \{A,B,C\}$, where U is the universal class.

BOUNDARY VALUE ANALYSIS:

It has been observed that programs that work correctly for a set of values in an equivalence class fails on some special values lie often on the boundary of the equivalence class.

In boundary value analysis, we choose an input for a test case from an equivalence class, such that the input lies at the edge of the equivalence classes. Boundary values for each equivalence classes of the output should be covered.

For example, Programmers may improperly use $<$, instead of $<=$, or conversely $<=$ instead of $<$. Boundary value analysis leads to selection of test cases at the boundaries of different equivalence classes.

CAUSE-EFFECT GRAPHICS:

One weakness with the equivalence class partitioning and boundary value methods is that they consider each input separately. Cause effect graphing is a technique that aids in selecting combination of input conditions in a systematic ways, such that the number of test cases does not become unmanageable large. The technique starts with identifying causes and effects of the system under testing. For example, an input condition can be "file is empty" which can be set to true by having empty input like and false by a non-empty file.

Structure Testing: -

STRUCTURAL TESTING | WHITE BOX TESTING | OR Glass Box Testing

In structure approaches, test cases are generated based on the actual code of the program or module to be tested.

This structure approach is some time called "Glass Box Testing."

The basis for deciding test cases in functional testing is the requirements or specifications of the system or module. For the entire system, the test cases are designed from the requirements specification document for the system, for modules created during design, test cases for functional testing are decided from the module specifications produced during the design.

The most obvious functional testing procedure is exhaustive testing, which as we have started is impractical.

Structural testing on the other hand is concerned with testing the implementation of the program. The intent of the structural testing is not to exercise all the different input or output conditions but to exercise the different programming structure and data structure used in the program.

To test the structure of the program, structure testing aims to achieve test cases that will force the designed coverage of different structure. Various criteria have been proposed for this. The criteria for functional testing. A white box testing strategy can either be coverage based or fault based.

1. Fault based: A fault based testing strategy targets to detect certain types of faults.

2. Coverage based : A popular example of coverage based testing strategy are statement coverage, branch coverage and the path coverage based testing.

1) STATEMENT COVERAGE

The simplest coverage criteria is statement coverage, which requires that each statement of the program to be executed at least once during testing. This coverage criteria is not very strong and can leave errors undetected.

It is difficult to determine whether it leads to a failure due to some illegal memory access. Wrong result computation due to improper arithmetic operation, etc.

2)BRACH COVERAGE:

Requires that each edge in the control flow graph be traversed at least once during testing.

In other words, Branch coverage requires that each decision in the program be evaluated to true and false values at least ones during testing. Testing based on branch coverage is often called branch testing.

3)PATH COVERAGE:

Path coverage requires that all possible paths in the control how graph be executed during testing.This is also known as all paths criterion and the testing.The difficulty with this criterion is that programs that contain loops can have an infinite number of possible paths.

Advantages of white Box testing:

- 1)As the knowledge of internal coding structure is prerequisite, it becomes very to find out which type of input data can help in testing the application effectively.
- 2)The otheradvantage of white box testing is helps in optimizing the code.
- 3)It helps in removing the extra lines of code, which can bring in hidden defects.

Disadvantages of white box testing:

- 1)As knowledge of code and internal structure is a prerequisite skilled tester is needed to carry out this type testing , which increases the cost.
- 2)And it is nearly impossible to look into every bit of code to find out hidden errors,which may create problems, resulting in failure of the application

Type of Testing: -

System testing consistes of three different kinds activities as follow:

- 1)**α-testing** :a testing is the system testing perfomed by the development team.

2)β-testing : This is the system testing performed by a customer.

3)Acceptance testing : This is the system testing performed by the customer himself

After the product is delivered to determine whether to accept the delivered product or to reject it.

4) Gamma testing is a little-known informal phrase that refers derisively to the release of “buggy” (defect-ridden) products. It is not a team of art among testers, but rather an example of referential humor. Cynics have referred to all software releases as “gamma testing” since defects are found in all commercial, commodity and publicity available software eventually.

Test Plan:-

Testing commences with a test plan and terminates with acceptance testing. A test plan is a general document for the entire project that defines the scope, approach to be taken and the schedule of testing as well as identifies the test items for the entire testing process and the personnel responsible for the different activities of testing.

There are three types of inputs for forming the test plan are:-

- 1) Project Plan
- 2) Requirement Documents
- 3) System Design Documents

A test plan should contain the following

- Test unit specification
- Features to be tested
- Approach for testing
- Test deliverables
- Schedule
- Personnel allocation

Test unit specification:-

- A test unit is a set of one or more modules, together with associated data, that are from a single computer program and that are the testing. A test unit can occur at any level and can contain from a single module to the entire system. Thus a test unit may be a module, or a complete system.

Features to be tested:-

- All software features and combination of features that should be tested. A Software features is a software characteristic specified or implied by the requirement or design documents. These may module functionality performance, design constraints and attributes.

Approach for testing:-

- The approach for testing specification the overall approach to be followed in the current project. The technique that will be judge the effort should be specified. This is sometimes called the testing criterion for evaluating and selecting test cases.

Test deliverables:-

- Testing deliverables should be specified in the test plan before the actual begins. Deliverables could be a list of test cases that were used detailed results of testing test summary report, test log, and data about the code coverage.
- In general a test case specification report, test summary and a test log should always be specified as deliverables.

Schedule:-

- The schedule specifies the amount of time and effort to be spent on different activities of testing and testing of different units that have been identified.

Personnel Allocation:-

- Personnel allocation identifies the person responsible for performing the different activities.

